

Sample Mock Questions

Easy

Q1. Given an positive integer n, print numbers from 1 to n without using loops. Implement the function printTillN() to print the numbers from 1 to n as space-separated integers.

(reference: <https://www.geeksforgeeks.org/dsa/how-will-you-print-numbers-from-1-to-200-without-using-loop/>)

Solution: If we take a look at this problem carefully, we can see that the idea of "loop" is to track some counter value, e.g., "i = 0" till "i <= 100". So, if we aren't allowed to use loops, how can we track something in the C language?

Well, one possibility is the use of 'recursion', provided we use the terminating condition carefully. Here is a solution that prints numbers using recursion.

Q2. Given an array **prices[]** of non-negative integers, representing the prices of the stocks on different days, return the maximum profit possible by buying and selling the stocks on different days when **at most one** transaction is allowed. Here one transaction means 1 buy + 1 Sell. If it is not possible to make a profit then **return 0**.

(reference: <https://www.geeksforgeeks.org/dsa/best-time-to-buy-and-sell-stock/>)

Note: Stock must be bought before being sold.

Examples :

Input: prices[] = [7, 10, 1, 3, 6, 9, 2]

Output: 8

Explanation: Buy for price 1 and sell for price 9.

Input: prices[] = [7, 6, 4, 3, 1]

Output: 0

Explanation: Since the array is sorted in decreasing order, 0 profit can be made without making any transaction.

Input: prices[] = [1, 3, 6, 9, 11]

Output: 10

Explanation: Since the array is sorted in increasing order, we can make maximum profit by buying at price[0] and selling at price[n-1]

Q3 You are given a huge integer a consisting of n digits ($1 \leq n \leq 3 \times 10^5$). The number may contain leading zeros.

You can perform the following operation any number of times:

Swap two adjacent digits only if they have different parity (i.e., one is even and the other is odd).

For example, if $a = 032867235$, you can make the following swaps:

Swap 0 and 3 $\rightarrow 302867235$

Swap 2 and 8 $\rightarrow 032876235$

Swap 7 and 2 $\rightarrow 032862735$

You cannot swap:

Non-adjacent digits.

Two adjacent digits having the same parity (both even or both odd).

Your task is to determine the smallest possible integer that can be obtained after performing any number of such operations.

Input

The first line contains an integer t — the number of test cases.

Each test case consists of a single string representing the integer a .

Constraints

$$1 \leq t \leq 10^4$$

$$1 \leq |a| \leq 3 \times 10^5$$

The sum of the lengths of all strings does not exceed 3×10^5 .

Output

For each test case, print the minimum possible integer after performing the allowed swaps.

REFERENCE:- Collated Mock Question - SET 2 [question 20]

Q4-Reverse and Add given number repeatedly to get a Palindrome number

Write a program that takes number and gives the resulting palindrome (if one exists). If it took more than 1, 000 iterations (additions) or yield a palindrome that is greater than 4, 294, 967, 295, assume that no palindrome exist for the given number.

Examples:

Input: N = 195

Output: 9339

Input: N = 265

Output: 45254

Input: N = 196

Output: No palindrome exists

REFERENCE:- <https://www.geeksforgeeks.org/dsa/reverse-and-add-function>

Medium

Q1. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. This means the first house is the neighbour of the last one. Meanwhile, adjacent houses have a security system connected, and it will automatically contact the police if two adjacent houses were broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example

- **Input:** `nums = [2, 3, 2]`
- **Output:** 3
- **Explanation:** You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses in a circular layout. Your best choice is to rob house 2 (money = 3).

Q2. Split an Array into Two Equal Sum Subarrays

Given an array of integers `arr[]`, determine whether it is possible to split it into two contiguous subarrays (without reordering the elements) such that the sum of the two subarrays is equal.

Input : `arr[] = [1 , 2 , 3 , 4 , 5 , 5]`

Output : true

Explanation : The array can be divided after index 3 into two subarrays: `[1, 2, 3, 4]` and `[5, 5]`.

Input : `arr[] = [4, 3, 2, 1]`

Output : false

Explanation: No possible split gives equal sum.

Q3. Given an array of n integers `nums` and a target integer `target`, find the number of index triplets (i, j, k) with $0 \leq i < j < k < n$ that satisfy the condition:

$$\text{nums}[i] + \text{nums}[j] + \text{nums}[k] < \text{target}$$

Example

- **Input:** `nums = [-2, 0, 1, 3]`, `target = 2`
- **Output:** 2
- **Explanation:** There are two triplets whose sum is less than 2:
 - `[-2, 0, 1]` (Sum = -1)
 - `[-2, 0, 3]` (Sum = 1)

Q4. A message containing letters from **A** to **Z** is encoded using the following mapping:

1 -> A
2 -> B
3 -> C
...
26 -> Z

Given a string `s` consisting only of digits, determine the total number of different ways it can be decoded.

Rules

- A single digit from '1' to '9' can be decoded as a letter.
- The digit '0' cannot be decoded by itself.
- '0' is valid only when it forms "10" or "20" with the previous digit.
- Two consecutive digits can be decoded together only if they form a number between **10** and **26** (inclusive).

Return the total number of possible decodings.

Example

Input : 226

Output : 3

Explanation

The possible decodings are:

- $2\ 2\ 6 \rightarrow \mathbf{BBF}$
- $22\ 6 \rightarrow \mathbf{VF}$
- $2\ 26 \rightarrow \mathbf{BZ}$

Hence the answer is 3.

Hard

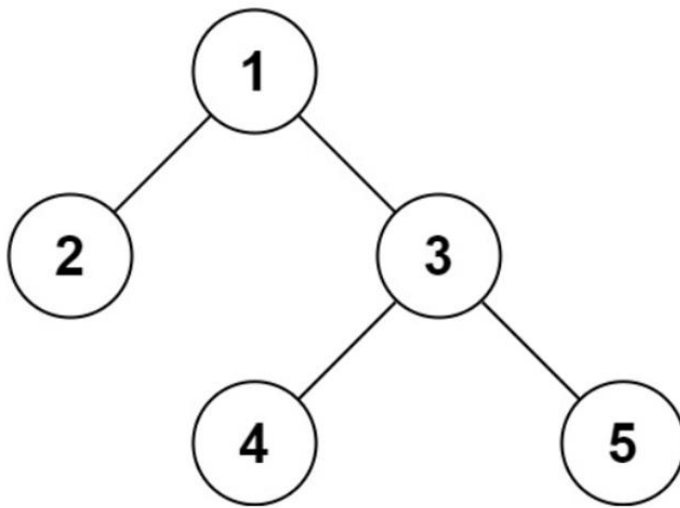
1.Serialize / deserialize binary tree

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The binary tree representation shown in the examples is for illustration purposes only. You may use any valid tree representation and solution approach in the mock test.

Example 1:



Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

2. Construct the Lexicographically Smallest Longest Palindromic Subsequence

Problem Statement

Given a string S consisting of uppercase English letters (A–Z), you are allowed to:

- Delete any number of characters from the string (including zero)
- **You are NOT allowed to reorder the remaining characters** — the relative order of the characters you keep must be preserved

Your task is to construct the **longest possible palindromic subsequence** using the given characters.

If there are multiple palindromic subsequences of the same maximum length, return the **lexicographically smallest** one among them.

Input Format

- A single string S

Output Format

- A single string representing the longest, lexicographically smallest palindromic subsequence of S

Constraints

- $1 \leq |S| \leq 1000$
- S consists of only uppercase English letters (A–Z)

Example

Case 1:

Input:

BBABCBCAB

Output:

BABCBAB

Explanation: The longest palindromic subsequence has length 7. Multiple subsequences of length 7 exist (e.g., different character choices), but BABCBAB is the lexicographically smallest among all of them.

Case 2:**Input:**

ABCDD

Output:

DD

Explanation: The longest palindromic subsequence has length 2. Both DD and single characters like A, B, C are palindromes, but only DD achieves the maximum length.

Case 3:**Input:**

CBABC

Output:

CBABC

Explanation: The entire string is already a palindrome and is trivially the longest one possible.

3. Sacred Realms of Eldoria

The kingdom of Eldoria was a land of unity, where its N sacred cities were connected through a vast network of enchanted bridges. Each city was a stronghold of knowledge and power.

However, an ancient sage shattered the harmony of Eldoria, causing the bridges to break randomly and leaving the kingdom divided into isolated regions.

The Great Oracle of Eldoria has foreseen a way to restore balance. He must study every possible subset of cities and count the total number of ways they can be split into exactly X isolated realms, where X is a prime number.

For every prime number X from 1 to N , the Oracle must:

- Determine the number of ways to choose a non-empty group of cities such that the bridges between them create exactly X separate connected realms.
- Sum up all prime values of X that contribute to the valid configurations.

Find the total number of valid ways for each prime number X from 1 to N .

Input Format

The first line contains an integer N , denoting the number of cities.

Each of the next N lines (where $1 \leq i \leq N$) contains an integer describing $Cities[i]$.

Constraints

- $1 \leq N \leq 5000$
- $1 \leq Cities[i] < i$

Sample Test Cases

Case 1

Input:

```
3
-1
1
1
```

Output:

```
1
```

Explanation: $N=3$, $Cities=[-1,1,1]$. For prime $X=2$: $result=1$. For prime $X=3$: $result=0$. Only subset $\{2,1\}$ forms exactly 2 connected components. Total = 1.

Case 2

Input:

4
-1
1
1
1

Output:

4

Explanation: $N=4$, $Cities=[-1,1,1,1]$. $X=2$, $result=3$ (subsets: (2,3), (2,4), (3,4)). $X=3$, $result=1$ (subset: (2,3,4)). Total=3+1=4.

Case 3

Input:

4
-1
1
2
1

Output:

5

Explanation: $N=4$, $Cities=[-1,1,2,1]$. $X=2$, $result=5$ (subsets: (1,3), (2,4), (3,4), (1,3,4), (2,3,4)). $X=3$, $result=0$. Total=5+0=5.

4. Water Connection Problem

Given n houses (numbered 1 to n) and p pipes connecting them, where each pipe is described by $a[i]$, $b[i]$, and $d[i]$ - a pipe of diameter $d[i]$ running from house $a[i]$ to house $b[i]$.

- Every house has at most one outgoing pipe and at most one incoming pipe.
- A house with an outgoing pipe but no incoming pipe gets a **tank** installed (the start of a chain).
- A house with an incoming pipe but no outgoing pipe gets a **tap** installed (the end of a chain).

Find every tank-tap pair, along with the minimum pipe diameter along the chain connecting them.

Note: Return a list of **[tank, tap, minimum diameter]** triples, sorted by tank number in ascending order.

Examples:

Input: $n = 9, p = 6, a[] = [7, 5, 4, 2, 9, 3], b[] = [4, 9, 6, 8, 7, 1], d[] = [98, 72, 10, 22, 17, 66]$

Output: $[[2, 8, 22], [3, 1, 66], [5, 6, 10]]$

Explanation: There are three separate chains: $3 \rightarrow 1, 5 \rightarrow 9 \rightarrow 7 \rightarrow 4 \rightarrow 6$, and $2 \rightarrow 8$. Tracing each from tank to tap: chain $3 \rightarrow 1$ has diameter 66, chain $5 \rightarrow 9 \rightarrow 7 \rightarrow 4 \rightarrow 6$ has a minimum diameter of 10 (the $4 \rightarrow 6$ pipe), and chain $2 \rightarrow 8$ has diameter 22.

Input: $n = 4, p = 2, a[] = [1, 3], b[] = [2, 4], d[] = [60, 50]$

Output: $[[1, 2, 60], [3, 4, 50]]$

Explanation: There are two separate chains: $1 \rightarrow 2$ and $3 \rightarrow 4$. Each chain has only one pipe, so its minimum diameter is just that pipe's diameter: 60 and 50 respectively.

Complex

Q1. Given an integer N , let us consider a triangle of numbers of N lines in which a number a_{11} appears in the first line, two numbers a_{21} and a_{22} appear in the second line, three numbers a_{31} , a_{32} and a_{33} appear in the third line, etc. In general, i numbers $a_{i1}, a_{i2}, \dots, a_{ii}$ appear in the i th line for all $1 \leq i \leq N$. Develop a program that will compute the largest of the sums of numbers that appear on the paths starting from the top towards the base, so that:

on each path the next number is located on the row below, more precisely either directly below or below and one place to the right.

Warning: large Input/Output data, be careful with certain languages

Input Format

The first line of the input contains an integer T , the number of test cases.

Then T test cases follow. Each test case starts with an integer N , the number of rows. Then N lines follow where in i th line contains i integers $a_{i1}, a_{i2}, \dots, a_{ii}$.

Output Format

For each test case print the maximum path sum in a separate line.

Constraints

$$1 \leq T \leq 1000$$

$$1 \leq N < 100$$

$$0 \leq a_{ij} < 100$$

Sample 1:

Input

2

21

123

4

12

412

2311

Output

5

9

Question 2: Magical Paths in the Infinite Forest (Complex)

Problem Statement

The **Infinite Forest** is represented as an $N \times M$ grid. You start your journey from the **top-left cell** $(0, 0)$ and want to reach the **bottom-right cell** $(N-1, M-1)$.

Each cell contains either:

- A **positive integer**, representing the magical cost required to enter that cell.
- **-1**, representing a **teleport tile**.

You are allowed to move **only right or down**.

Teleport Rule

- If you enter a teleport tile (-1) , you may instantly **teleport to any other teleport tile** in the grid.
- Teleportation is **free**.
- You **do not pay any cost** for the destination teleport tile.
- However, you must first reach a teleport tile through normal movement before using teleportation.

Your task is to determine the **minimum magical cost** required to travel from the starting cell to the destination while making optimal use of teleportation.

Input Format

- The first line contains two integers **N** and **M**, representing the number of rows and columns.
- The next **N** lines each contain **M** space-separated integers representing the grid.
 - A positive integer denotes the cost of entering that cell.
 - -1 denotes a teleport tile.

Output Format

Print a single integer representing the **minimum magical cost** to reach the bottom-right cell.

Constraints

- $1 \leq N, M \leq 100$
- Cell value is either -1 or a positive integer.
- Movement is allowed **only to the right or downward**.

Sample Input 1

```
4 4
1 3 -1 4
2 8 5 9
4 -1 3 1
6 7 2 5
```

Sample Output 1

```
13
```

Sample Input 2

```
3 3
1 -1 3
2 4 -1
1 1 1
```

Sample Output 2

```
2
```

Test Cases

Input 1

```
3 3
1 2 3
4 5 6
7 8 9
```


Output 1

21

Input 2

3 3

1 -1 3

2 100 -1

1 1 1

Output 2

2

Input 3

4 4

1 2 -1 9

8 9 9 9

-1 9 9 9

9 9 9 1

Output 3

31

Input 4

3 3

-1 3 1

2 -1 2

3 4 1

Output 4

3

Input 5

3 3
1 2 3
4 5 6
7 8 -1

Output 5

12

Input 6

3 3
1 2 3
4 -1 6
7 8 9

Output 6

18

Input 7

4 4
1 -1 -1 1
1 100 100 1
1 100 100 1
1 1 1 1

Output 7

5

Input 8

4 4

1 99 99 99

-1 99 99 99

99 99 99 -1

99 99 99 1

Output 8

2

Question 3: Number of Bases Where N Starts With 1

Problem Statement

Altaf has recently learned about number bases and is becoming fascinated.

Altaf learned that for bases greater than ten, new digit symbols need to be introduced, and that the convention is to use the first few letters of the English alphabet. For example, in base 16, the digits are 0123456789ABCDEF. Altaf thought that this is unsustainable; the English alphabet only has 26 letters, so this scheme can only work up to base 36. But this is no problem for Altaf, because Altaf is very creative and can just invent new digit symbols when she needs them. (Altaf is very creative.)

Altaf also noticed that in base two, all positive integers start with the digit 1! However, this is the only base where this is true. So naturally, Altaf wonders: Given some integer N , how many bases b are there such that the base- b representation of N starts with a 1?

INPUT FORMAT:

The first line of the input contains an integer T denoting the number of test cases. The description of T test cases follows.

Each test case consists of one line containing a single integer N (in base ten).

OUTPUT FORMAT:

For each test case, output a single line containing the number of bases b , or INFINITY if there are an infinite number of them.

CONSTRAINTS :

$$1 \leq T \leq 10^5$$

$$0 \leq N < 10^{12}$$

Sample Input:-

4

6

9

11

24

Sample Output:- 4

7

8

14